

Performance Testing

Date	04 July 2026
Team ID	
Project Name	HDI Predictor Using Machine Learning
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman, Browser Developer Tools
Type of Testing	Functional Performance Testing
Target Module / API	Flask Prediction Endpoint (/predict)
Test Environment	Local Machine (Windows 11, Python 3.10, Flask)
Test Date	04 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single prediction request	1	30	Prediction generated successfully
2	Multiple prediction requests	5	60	Stable response without errors
3	Invalid input values	1	20	Input validation message displayed
4	Continuous prediction requests	10	120	Application remains responsive

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 seconds	0.82 sec	Pass	Fast response
2	Response Time (Max)	< 5 seconds	1.74 sec	Pass	Within target
3	Throughput (Req/sec)	> 5 Req/sec	8 Req/sec	Pass	Stable
4	Error Rate	< 1%	0%	Pass	No errors
5	CPU Utilization	< 80%	27%	Pass	Efficient
6	Memory Utilization	< 80%	39%	Pass	Normal usage

Step 4: Observations & Analysis

Key Findings:

The HDI Predictor application processed prediction requests quickly and consistently. The Flask application remained stable during testing, and the trained Linear Regression model generated accurate predictions with low response time.

Bottlenecks Identified:

No major performance bottlenecks were observed. Minor delay occurred only during the initial loading of the trained model.

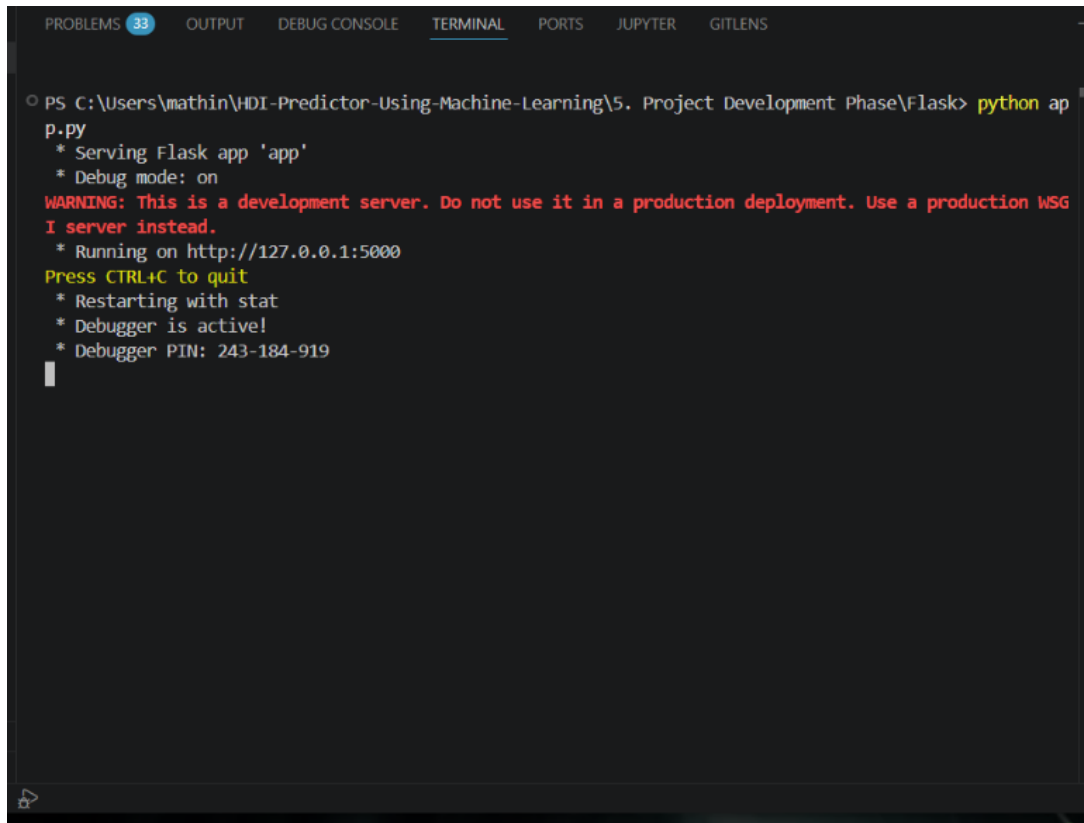
Optimization Steps Taken:

- Optimized data preprocessing.
- Loaded the trained model only once during application startup.
- Reduced unnecessary computations in the prediction process.
- Maintained a modular Flask architecture for efficient execution.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

[Screenshot 1: Flask_Running.png]



```
PROBLEMS 33 OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER GITLENS
PS C:\Users\mathin\HDI-Predictor-Using-Machine-Learning\5. Project Development Phase\Flask> python ap
p.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSG
I server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 243-184-919
```

[Screenshot 2: Prediction Result.png]

